Конспект лекции 2: Свёрточные нейронные сети

Липкин С.М.

1 Мотивация

Полносвязные (FC) слои плохо подходят для обработки изображений по нескольким причинам:

- Огромное число параметров: изображение $224 \times 224 \times 3$ даёт 150 528 входных нейронов. Один скрытый слой на 1024 нейрона — 150 миллионов параметров.
- Нет инвариантности к сдвигу: кошка в левом углу и кошка в правом углу — разные активации.
- Нет локальности: каждый пиксель влияет на каждый нейрон.

Свёрточные нейронные сети (CNN) решают эти проблемы через:

- **Локальную связность** (local connectivity): каждый нейрон видит только локальную область изображения (receptive field).
- **Разделение весов** (weight sharing): один и тот же фильтр применяется ко всем участкам. Это даёт инвариантность к сдвигу и в 10–100 раз меньше параметров.
- **Иерархию признаков**: первые слои учат края и текстуры, следующие — формы и части объектов.

2 Что такое свёртка

2.1 Математическое определение

Дискретная 1D свёртка:

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m] \cdot g[n-m]$$

На практике в CNN используется **кросс-корреляция** (ядро не переворачивается):

$$S[i,j] = \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} I[i+u, j+v] \cdot K[u,v]$$

где:

- I — входной сигнал (изображение, feature map) размера $H_{\text{in}} \times W_{\text{in}}$
- K — ядро (kernel, filter) размера $K_h \times K_w$
- S — выходной сигнал (feature map)

2.2 Интуиция

Ядро «скользит» по изображению с шагом S (stride). В каждой позиции вычисляется скалярное произведение между ядром и локальной областью изображения. Результат — карта активаций (feature map), показывающая, **где** ядро «нашло» искомый паттерн.

Если ядро — детектор горизонтальных краев, то feature map будет иметь высокие значения там, где есть горизонтальные границы.

3 Свёрточная арифметика

3.1 Основная формула

Размер выхода свёрточного слоя:

$$O = \left\lfloor \frac{I + 2P - K}{S} \right\rfloor + 1$$

где:

- I — размер входа (по одному измерению)
- K — размер ядра
- P — padding (количество нулевых пикселей по краям)
- S — stride (шаг)
- O — размер выхода

Если по разным осям padding или stride разный, формула применяется отдельно к каждой оси:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P_h - K_h}{S_h} \right\rfloor + 1$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P_w - K_w}{S_w} \right\rfloor + 1$$

3.2 Типы padding

- **Valid** ($P = 0$): $O = (I - K)/S + 1$ — выход сжимается.
- **Same** ($P = \lfloor K/2 \rfloor$): $O = I/S$ — размер сохраняется (при $S = 1$).
- **Full** ($P = K - 1$): $O = I + K - 1$ — вход и ядро полностью перекрываются хотя бы на одном пикселе.

3.3 Примеры

- $I = 32$, $K = 5$, $S = 1$, Valid: $O = 32 - 5 + 1 = 28$
- $I = 32$, $K = 5$, $S = 1$, Same: $O = 32$ (при $P = 2$)
- $I = 32$, $K = 3$, $S = 2$, Valid: $O = \lfloor (32 - 3)/2 \rfloor + 1 = 15$

3.4 Многоканальная свёртка

Вход: $H \times W \times C_{\text{in}}$ (например, RGB: 3 канала).

Ядро: $K_h \times K_w \times C_{\text{in}} \times C_{\text{out}}$. Каждый выходной канал — сумма свёрток по всем входным каналам:

$$S_j = \sum_{i=1}^{C_{\text{in}}} I_i * K_{ij}$$

Количество параметров слоя:

$$\text{params} = K_h \cdot K_w \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$$

3.5 Receptive Field

Размер области входа, «видимой» одним нейроном:

$$RF_l = RF_{l-1} + (K_l - 1) \cdot \prod_{i=1}^{l-1} S_i$$

Начальное значение: $RF_0 = 1$.

4 Базовые блоки CNN

4.1 Pooling (субдискретизация)

Max Pooling:

$$y_{ij} = \max_{u=1..K, v=1..K} x_{iS+u, jS+v}$$

Выделяет наиболее активный признак, инвариантен к малым сдвигам. Нет обучаемых параметров.

Average Pooling:

$$y_{ij} = \frac{1}{K^2} \sum_{u=1..K, v=1..K} x_{iS+u, jS+v}$$

Более «гладкий» выход.

Global Pooling: pooling по всему пространству ($K = H = W$) $\to 1 \times 1$.

Зачем: уменьшение размерности $\to$ меньше параметров $\to$ bigger receptive field без роста вычислений.

4.2 Транспонированная свёртка (Deconvolution)

Операция увеличения пространственного разрешения. Если обычная свёртка — от многих к одному, то транспонированная — от одного ко многим.

Формула:

$$O = (I - 1) \cdot S + K - 2P$$

Важно: это **не** обратная свёртка. Это fractionally-strided convolution. Используется в генеративных моделях (GAN, VAE), сегментации (U-Net) и super-resolution.

4.3 Dilated (Atrous) Convolution

Увеличивает receptive field без увеличения числа параметров:

$$K_{\text{effective}} = K + (K - 1) \cdot (d - 1)$$

где d — dilation rate. Для 3×3 ядра:

- $d = 1$: $K_{\text{eff}} = 3$, RF $= 3 \times 3$
- $d = 2$: $K_{\text{eff}} = 5$, RF $= 5 \times 5$
- $d = 4$: $K_{\text{eff}} = 9$, RF $= 9 \times 9$

Параметров не прибавляется. Используется в DeepLab для семантической сегментации.

4.4 Depthwise Separable Convolution

Разделяет стандартную свёртку на два этапа:

1. **Depthwise:** $K \times K$ свёртка для каждого канала отдельно: $\text{params}_{\text{dw}} = K^2 \cdot C_{\text{in}}$
2. **Pointwise (1×1):** линейная комбинация каналов: $\text{params}_{\text{pw}} = C_{\text{in}} \cdot C_{\text{out}}$

Сравнение: для 3×3, $C_{\text{in}} = 64$, $C_{\text{out}} = 128$:

- Обычная: $3^2 \cdot 64 \cdot 128 = 73\,728$ параметров
- Depthwise Separable: $3^2 \cdot 64 + 64 \cdot 128 = 8\,768$ параметров
- Ускорение: **8.4x**

Используется в MobileNet, Xception, EfficientNet.

4.5 1×1 Свёртка (Pointwise Convolution)

Линейная комбинация каналов без пространственного взаимодействия:

$$y_{ij}^{(k)} = \sum_{c=1}^{C_{\text{in}}} w_c^{(k)} \cdot x_{ij}^{(c)}$$

Применения: изменение числа каналов (bottleneck), снижение размерности перед дорогими 3×3 свёртками.

Bottleneck в ResNet: 1×1 ($256 \to 64$) $\to$ 3×3 ($64 \to 64$) $\to$ 1×1 ($64 \to 256$). 3×3 работает в 4 раза быстрее (64 канала вместо 256).

5 Архитектуры CNN

5.1 LeNet-5 (LeCun, 1998)

Первая успешная CNN. Для распознавания рукописных цифр MNIST.

Архитектура:

- Вход: 32×32 grayscale

- Conv 5×5, 6 каналов $\to$ AvgPool 2×2
- Conv 5×5, 16 каналов $\to$ AvgPool 2×2
- Conv 5×5, 120 каналов (фактически FC)
- FC 84 $\to$ FC 10 (softmax)

Всего 60K параметров. Паттерн «свёртка $\to$ пулинг $\to$ свёртка $\to$ пулинг $\to$ FC» стал шаблоном для всех последующих CNN.

5.2 AlexNet (Krizhevsky, 2012)

Победитель ImageNet 2012. Переломный момент для DL.

Ключевые инновации:

- **ReLU** вместо tanh — решил проблему исчезающего градиента.
- **Data augmentation** — сдвиги, отражения, изменение цвета.
- **Dropout** (0.5 на FC-слоях) — борьба с overfitting.
- Local Response Normalization (устарело, заменено BatchNorm).
- Обучение на 2 GPU параллельно.

Архитектура: 5 свёрточных + 3 FC слоя. Conv: 11×11 (96 каналов), 5×5 (256), 3×3 (384, 384, 256). MaxPool 3×3 после первых двух и последней свёртки. FC: 4096 $\to$ 4096 $\to$ 1000.

62M параметров. Время обучения: 5–6 дней. Запустил эру глубокого обучения.

5.3 VGG (Simonyan & Zisserman, 2014)

Главная идея: стек из 3×3 свёрток. Два 3×3 слоя $\to$ RF 5×5. Три 3×3 слоя $\to$ RF 7×7.

Почему это выгоднее одного большого ядра:

- 3×3: 9 параметров на канал.
- 5×5: 25 параметров на канал (2.78$\times$ больше).
- 7×7: 49 параметров на канал (5.44$\times$ больше).
- Дополнительные нелинейности между слоями.

VGG16: Conv 3×3: 64 $\to$ 128 $\to$ 256 $\to$ 512 $\to$ 512, MaxPool 2×2 после каждой группы, 3 FC слоя. 138M параметров. Показал, что глубина важнее ширины.

5.4 Inception / GoogLeNet (Szegedy, 2014)

Проблема: какой размер ядра выбрать?

Решение Inception module: используем **все сразу** — параллельные ветки с 1×1, 3×3, 5×5 свёртками и 3×3 MaxPool, конкатенация по каналам.

1×1 свёртки перед дорогими 3×3 и 5×5 играют роль bottleneck — снижают размерность.

Результат: 6.8M параметров (VGG: 138M) при сопоставимом качестве. Использовал auxiliary classifiers для борьбы с исчезающим градиентом.

5.5 ResNet (He et al., 2015)

Проблема: при > 20 слоях качество падает (degradation problem). Не из-за overfitting — и train, и test loss растут.

Решение: Residual Learning

$$H(x) = F(x) + x$$

Слой учится **добавке** (residual) $F(x)$ к тождественному отображению x. Если слой не нужен: $F(x) \to 0$, $H(x) \to x$. Градиент течёт через shortcut напрямую.

Влияние на градиент:

$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_{l+1}} \left(1 + \frac{\partial F(x_l)}{\partial x_l}\right)$$

Единичная матрица сохраняет градиент — vanishing gradient практически устранён.

Bottleneck block: 1×1 $(256 \to 64) \to 3 \times 3$ $(64 \to 64) \to 1 \times 1$ $(64 \to 256)$.

Результат: 152 слоя, 3.57% top-5 error на ImageNet. «Рабочая лошадка» до сих пор.

5.6 DenseNet (Huang et al., 2017)

Идея: каждый слой получает на вход feature maps со **всех предыдущих слоёв**:

$$x_l = H_l([x_0, x_1, \ldots, x_{l-1}])$$

Преимущества:

- Максимальный поток информации и градиентов — каждый слой имеет прямой доступ к градиенту лосса.
- Естественная регуляризация — нет дублирующих признаков.
- Меньше параметров, чем у ResNet при схожем качестве (DenseNet-121: 8M vs ResNet-50: 25M).

Недостаток: большой расход памяти (нужно хранить все активации для backpropagation).

5.7 MobileNet (2017–2018)

Оптимизированы для мобильных устройств:

- Depthwise Separable Convolution везде.
- Width multiplier α — пропорциональное уменьшение числа каналов.
- Resolution multiplier ρ — уменьшение разрешения входа.

MobileNetV2: Inverted Residual (узкий $\to$ широкий $\to$ узкий) + Linear bottleneck.

5.8 EfficientNet (Tan & Le, 2019)

Ключевая идея: композитное масштабирование. Обычно масштабируют **один** параметр (глубину, ширину или разрешение). EfficientNet масштабирует **все три** сбалансированно:

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi$$

где $\alpha\beta^2\gamma^2 \approx 2$ (FLOPs растут в 2^ϕ раз).

EfficientNet-B7: 66M параметров, достигает SOTA при **в 10x меньше FLOPs**, чем предыдущие SOTA.

6 Сводка архитектур

Архитектура	**Год**	**Парам.**	**Top-5 err.**	**Ключевая идея**
LeNet-5	1998	60K	—	Первая CNN
AlexNet	2012	62M	16.4%	ReLU, Dropout, GPU
VGG16	2014	138M	8.8%	3×3 стеки
GoogLeNet	2014	6.8M	6.7%	Inception modules
ResNet-152	2015	60M	3.6%	Skip connections
DenseNet-121	2017	8M	5.6%	Dense connections
MobileNetV2	2018	3.5M	—	Depthwise sep.
EfficientNet-B7	2019	66M	2.2%	Compound scaling

7 Заключение

Эволюция архитектур CNN: LeNet задал паттерн, AlexNet показал масштаб, VGG — глубину, Inception — ширину, ResNet решил degradation problem, DenseNet максимизировал поток информации, MobileNet оптимизировал для устройств, EfficientNet нашёл баланс масштабирования.

Несмотря на доминирование Transformer в NLP и Vision (ViT), CNN остаются основой для множества приложений: детекция, сегментация, генеративные модели, обработка изображений на устройствах.